Introduction to Computer Organisation and Architecture UCCD1133

Chapter 4

Computer Architecture and Organization Fundamental

Disclaimer

- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Outline

- Programming

MIPS Programming –Part I

- High-level languages:
  - e.g., C, Java, Python
- Written at higher level of abstraction
- Common high-level software constructs:
  - Arithmetic
  - Logical operations
  - if/else statements, for loops, while loops
  - Arrays indexing
  - function calls

Instruction - Logical

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

- **AND, OR, XOR, NOR**
  - AND: useful for masking bits
  - Masking all but the least significant byte of a value:

    0xF234012F AND 0x000000FF = 0x0000002F
  - OR: useful for combining bit fields. Combine 0xF2340000 with 0x000012BC:

    0xF2340000 OR 0x000012BC = 0xF23412BC
  - NOR: useful for inverting bits:

    A NOR $0 = NOT A

- **andi, ori, xori**
- 16-bit immediate is zero-extended (not sign-extended)

Instruction - Logical

Example 1:

Source Registers

$s1	1111	1111	1111	1111	0000	0000	0000	0000
$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code	Result								
`and $s3, $s1, $s2`	$s3								
`or  $s4, $s1, $s2`	$s4								
`xor $s5, $s1, $s2`	$s5								
`nor $s6, $s1, $s2`	$s6								

Example 2:

Source Values

$s1	0000	0000	0000	0000	0000	0000	1111	1111
imm	0000	0000	0000	0000	1111	1010	0011	0100

zero-extended

Assembly Code	Result								
`andi $s2, $s1, 0xFA34`	$s2								
`ori  $s3, $s1, 0xFA34`	$s3								
`xori $s4, $s1, 0xFA34`	$s4								

Solution:

Source Registers

$s1	1111	1111	1111	1111	0000	0000	0000	0000
$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code	Result								
`and $s3, $s1, $s2`	$s3	0100	0110	1010	0001	0000	0000	0000	0000
`or  $s4, $s1, $s2`	$s4	1111	1111	1111	1111	1111	0000	1011	0111
`xor $s5, $s1, $s2`	$s5	1011	1001	0101	1110	1111	0000	1011	0111
`nor $s6, $s1, $s2`	$s6	0000	0000	0000	0000	0000	1111	0100	1000

Solution:

Source Values

$s1	0000	0000	0000	0000	0000	0000	1111	1111
imm	0000	0000	0000	0000	1111	1010	0011	0100

zero-extended

Assembly Code	Result								
`andi $s2, $s1, 0xFA34`	$s2	0000	0000	0000	0000	0000	0000	0011	0100
`ori  $s3, $s1, 0xFA34`	$s3	0000	0000	0000	0000	1111	1010	1111	1111
`xori $s4, $s1, 0xFA34`	$s4	0000	0000	0000	0000	1111	1010	1100	1011

Instruction - Shift

- **sll (shift left logical)**
  - Shift left and fill with 0 bits

Example: sll $t0, $t1, 5 # $t0 <= $t1 << 5

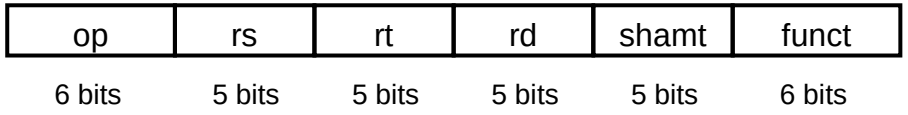
- **srl (shift right logical)**
  - Shift right and fill with 0 bits

Example: srl $t0, $t1, 5 # $t0 <= $t1 >> 5

- **sra (shift right arithmetic)**

Example: sra $t0, $t1, 5 # $t0 <= $t1 >>> 5

- shamt: how many positions to shift



shift left logical	`sll rd, rt, shamt`
shift right arithmetic	`sra rd, rt, shamt`
shift right logical	`srl rd, rt, shamt`

Assembly Code / Field Values

Assembly Code	op	rs	rt	rd	shamt	funct
`sll $t0, $s1, 2`	0	0	17	8	2	0
`srl $s2, $s1, 2`	0	0	17	18	2	2
`sra $s3, $s1, 2`	0	0	17	19	2	3
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00010	000000	(0x00114080)
000000	00000	10001	10010	00010	000010	(0x00119082)
000000	00000	10001	10011	00010	000011	(0x00119883)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Instruction - Shift

Example 3

```
sll $t0, $s1, 4

srl $s2, $s1, 4

sra $s3, $s1, 4
```

Field Values

op	rs	rt	rd	shamt	funct
0	0	17	8	4	0
0	0	17	18	4	2
0	0	17	19	4	3
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00100	000000	(0x00114100)
000000	00000	10001	10010	00100	000010	(0x00119102)
000000	00000	10001	10011	00100	000011	(0x00119903)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Solution

Assembly Code	Result							
sll $t0, $s1, 4	$t0	0011	0000	0000	0000	0010	1010	1000	0000
srl $s2, $s1, 4	$s2	0000	1111	0011	0000	0000	0000	0010	1010
sra $s3, $s1, 4	$s3	1111	1111	0011	0000	0000	0000	0010	1010

Assume

Source Values

$s1	1111	0011	0000	0000	0000	0010	1010	1000

Generating constant

- The addi instruction is helpful for assigning 16-bit constants

High-Level Code	MIPS Assembly Code
`int a = 0x4f3c;`	`# $s0 = a` `addi $s0, $0, 0x4f3c  # a = 0x4f3c`

The `int` data type in C refers to a word of data representing a two's complement integer. MIPS uses 32-bit words, so an `int` represents a number in the range $[-2^{31}, 2^{31}-1]$.

- To assign 32-bit constants:
  - use a load upper immediate instruction (lui)
  - followed by an or immediate (ori) instruction

High-Level Code	MIPS Assembly Code

`int a = 0x6d5e4f3c;`	`# $s0 = a`
	`lui $s0, 0x6d5e  # a = 0x6d5e0000`
	`ori $s0, $s0, 0x4f3c  # a = 0x6d5e4f3c`

- Where lui loads a 16-bit immediate into the upper half of a register and sets the lower half to 0.

Multiplication & division

- **Multiplication and division are different from other arithmetic operations.**
- Multiplying two 32-bit numbers produces a 64-bit product.
- Dividing two 32-bit numbers produces a 32-bit quotient and a 32-bit remainder.
- The MIPS architecture has **two special-purpose registers** (which are used to hold the results of multiplication and division):
  - hi
  - lo
- **mult $s0, $s1** # $s0 X $s1
  - The **32 most significant bits** of the product are placed in **hi**
  - the **32 least significant bits** are placed in **lo.**
- **div $s0, $s1** # $s0/$s1.
  - The **quotient** is placed in **lo**
  - the **remainder** is placed in **hi.**
- MIPS provides another multiply instruction that produces a 32-bit result in a general-purpose register.
- **mul $s1, $s2, $s3** #$s1= $s2 X $s3
- hi and lo are not among the usual 32 MIPS registers, so special instructions are needed to access these two registers.
- mfhi $s2 #move from hi
  - copies the value in hi to $s2.
- mflo $s3 #move from lo
  - copies the value in lo to $s3.

Branching

- Execute instructions out of sequence
- Types of branches:

1. Conditional

- Branch to a labeled instruction if a condition is true. Otherwise, continue sequentially.
- branch if equal (beq rs rt L1)
  - If (rs == rt) branch to instruction labeled L1
- branch if not equal (bne rs rt L1)
  - If (rs != rt) branch to instruction labeled L1

2. Unconditional

- jump (j)
- jump register (jr)
- jump and link (jal)

1. Conditional Branching

- The MIPS instruction set has two conditional branch instructions:
  - branch if equal (beq)
  - branch if not equal (bne).

Branching (Conditional)

Example 1:

MIPS Assembly Code

```
addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
addi  $s1, $0, 1          # $s1 = 0 + 1 = 1
sll   $s1, $s1, 2         # $s1 = 1 << 2 = 4
beq   $s0, $s1, target    # $s0 == $s1, so branch is taken
addi  $s1, $s1, 1         # not executed
sub   $s1, $s1, $s0       # not executed

target:                   #label
add   $s1, $s1, $s0       # $s1 = 4 + 4 = 8
```

- **Labels** indicate instruction location. They can't be reserved words and must be followed by colon (:)
- Branches are written as

  beq rs, rt, imm

  where rs is the first source register. **This order is reversed from most I-type instructions**

- Assembly code uses labels to indicate instruction locations in the program.
- When the assembly code is translated into machine code, these labels are translated into instruction addresses

Example 2:

MIPS Assembly Code

```
addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
addi  $s1, $0, 1          # $s1 = 0 + 1 = 1
sll   $s1, $s1, 2         # $s1 = 1 << 2 = 4
bne   $s0, $s1, target    # $s0 == $s1, so branch is not taken
addi  $s1, $s1, 1         # $s1 = 4 + 1 = 5
sub   $s1, $s1, $s0       # $s1 = 5 – 4 = 1

target:
add   $s1, $s1, $s0       # $s1 = 1 + 4 = 5
```

Jump (Unconditional)

- Jump (j) jumps directly to the instruction at the specified label.

MIPS Assembly Code

```
  addi  $s0, $0, 4      # $s0 = 4
  addi  $s1, $0, 1      # $s1 = 1
  j     target          # jump to target
  addi  $s1, $s1, 1     # not executed
  sub   $s1, $s1, $s0   # not executed

target:
  add   $s1, $s1, $s0   # $s1 = 1 + 4 = 5
```

Conditional Statement

- Statements are conditional statements commonly used in high-level languages:
  - if statements
  - if/else statements
  - while loops
  - for loops

1. If statements

High-Level Code

```
if (i == j)
  f = g + h;

f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
  bne $s3, $s4, L1    # if i != j, skip if block
  add $s0, $s1, $s2   # if block: f = g + h
L1:
  sub $s0, $s0, $s3   # f = f - i
```

***Assembly tests opposite case (i != j) of high-level code (i == j)**

If/else ststement

- if/else statements execute one of two blocks of code depending on a condition.
  - When the condition in the if statement is met, the if block is executed.
  - Otherwise, the else block is executed.

High-Level Code

```
if (i == j)
  f = g + h;

else
  f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
  bne $s3, $s4, else   # if i != j, branch to else
  add $s0, $s1, $s2    # if block: f = g + h
  j   L2               # skip past the else block
else:
  sub $s0, $s0, $s3    # else block: f = f - i
L2:
```

- The assembly code tests for the opposite condition (i!= j).
  - If that opposite condition is TRUE, bne skips the if block and executes the else block.
  - Otherwise, the if block executes and finishes with a jump instruction (j) to jump past the else block.

While loop

- while loops repeatedly execute a block of code until a condition is not met.

High-Level Code

```
int pow = 1;
int x = 0;


while (pow != 128)
{
  pow = pow * 2;
  x = x + 1;
}
```

MIPS Assembly Code

```
# $s0 = pow, $s1 = x
  addi $s0, $0, 1       # pow = 1
  addi $s1, $0, 0       # x = 0

  addi $t0, $0, 128     # t0 = 128 for comparison
while:
  beq  $s0, $t0, done   # if pow == 128, exit while loop
  sll  $s0, $s0, 1      # pow = pow * 2
  addi $s1, $s1, 1      # x = x + 1
  j    while
done:
```

Assembly tests for the opposite case (pow == 128) of the C code (pow != 128).

For loop

- for loops, like while loops, repeatedly execute a block of code until a condition is not met.
- for loops add support for a loop variable, which typically keeps track of the number of loop executions. A general format of the for loop is

  for (initialization; condition; loop operation)

  statement

  where
  - initialization: executes before the loop begins
  - condition: is tested at the beginning of each iteration
  - loop operation: executes at the end of each iteration
  - statement: executes each time the condition is met

High-Level Code

```
int sum = 0;



for (i = 0; i != 10; i = i + 1) {
  sum = sum + i ;


}

// equivalent to the following while loop
int sum = 0;
int i = 0;
while (i != 10) {
  sum = sum + i;
  i = i + 1;
}
```

MIPS Assembly Code

```
# $s0 = i, $s1 = sum
  add  $s1, $0, $0      # sum = 0
  addi $s0, $0, 0       # i = 0
  addi $t0, $0, 10      # $t0 = 10

for:
  beq  $s0, $t0, done   # if i == 10, branch to done
  add  $s1, $s1, $s0    # sum = sum + i
  addi $s0, $s0, 1      # increment i
  j    for

done:
```

More Conditional Operation

- Set result to 1 if a condition is true, Otherwise, set to 0

- slt rd, rs, rt

  if (rs < rt) rd = 1; else rd = 0

- slti rt, rs, constant

  if (rs < constant) rt = 1; else rt = 0

- Use in combination with beq, bne

```
slt $t0, $s1, $s2       # if ($s1 < $s2)
bne $t0, $zero, L       #   branch to L
```

Q &A